

Trabajo Práctico — Unidad 3, Semana 5

Índices, vistas y vistas materializadas en Food Store

Informe de entrega — Flujo con Kiro, OpenCode y Git

Materia	Base de Datos II
Unidad / Semana	Unidad 3 — Semana 5
Proyecto base	Food Store (continúa el esquema de las Semanas 1 a 4: schema.sql, data.sql, queries.sql)
Motor de BD	PostgreSQL 16.2 (mediciones reales en base dedicada)
Herramientas	OpenCode (generación en terminal), Kiro (especificaciones), Git/GitHub
Modalidad	Equipo: Gómez — Pappalardo — Ibáñez — Arroyo — Reinoso — Sisterna
Entrega	Repositorio Git + bitácora de IA (DUIA) + informe de mediciones
Contenido	indices.sql (Parte A) + views.sql (Partes B y C) + specs/ + duia.md + informe_mediciones.md

1. Presentación del trabajo

Este informe documenta el trabajo de la Semana 5: plan de indexado justificado con mediciones EXPLAIN ANALYZE antes y después, tres vistas (una con criterio de seguridad) y una vista materializada para el reporte agregado más costoso. Se trabaja sobre el esquema Food Store heredado (categoría, producto, usuario, pedido, detalle_pedido) sin modificar tablas base: solo se agregan índices y vistas.

Metodología: PostgreSQL 16.2, base dedicada con volumen (5004 pedidos vigentes, 10007 detalles vigentes, 2014 productos vigentes), VACUUM ANALYZE previo, cada EXPLAIN (ANALYZE, BUFFERS) ejecutado 3 veces y se informa la mediana. Script reproducible: mediciones.sql. Orden: medir ANTES (solo índices de Semana 3), crear indices.sql, medir DESPUÉS. Q1 usa BETWEEN CURRENT_DATE - 365 AND CURRENT_DATE (medición: septiembre 2026).

Flujo IA exigido por la cátedra: especificar primero en Kiro (carpeta specs/, 8 specs), generar con OpenCode, leer línea por línea, probar en transacción reversible y versionar en commits separados. La decisión nunca se delega.

2. Parte A — Plan de indexado (indices.sql)

A1 — Q1: pedidos confirmados de un año (642 filas)

Consulta: SELECT id_pedido, fecha_pedido, total_pedido FROM pedido WHERE fecha_pedido BETWEEN CURRENT_DATE - 365 AND CURRENT_DATE AND estado_pedido = 'CONFIRMADO' AND eliminado = FALSE. Índice: idx_pedido_estado_fecha (estado_pedido, fecha_pedido) WHERE eliminado = FALSE. Regla B-tree: igualdad antes que rango; parcial por borrado lógico.

ANTES: Seq Scan on pedido, filtra 4363 filas, Buffers hit=150, Execution Time mediana 0.258 ms (0.364 / 0.255 / 0.258). DESPUÉS: Bitmap Heap Scan + Bitmap Index Scan on idx_pedido_estado_fecha (Heap

Blocks exact=36, hit=39), mediana 0.084 ms (0.130 / 0.084 / 0.078). Mejora ~3x, -74% buffers. Cambio de plan: Sí. Cumple en plan y buffers; en tiempo no llega a 10x porque el dataset cabe en caché.

```
CREATE INDEX IF NOT EXISTS idx_pedido_estado_fecha
ON pedido (estado_pedido, fecha_pedido)
WHERE eliminado = FALSE;
```

A2 — Q2: Top 5 productos más vendidos (resultado honesto: SIN cambio)

Consulta: agregado de queries.sql A (JOIN + GROUP BY producto_id sobre 10007 filas). Índice: idx_detalle_producto (producto_id) WHERE eliminado = FALSE. ANTES: Seq Scan + Hash Join + HashAggregate, 2.203 ms (2.203 / 2.140 / 2.222). DESPUÉS: mismo plan, 2.177 ms (2.236 / 2.177 / 2.133). Sin cambio y es lo esperado: un agregado TOTAL debe leer las 10007 filas igual; ningún B-tree evita ese escaneo. El índice se CONSERVA porque su caso de uso es el acceso SELECTIVO (ver A2b) y los JOINS filtrados.

A2b — Complementaria selectiva: ventas de un producto (770 filas)

Consulta: SELECT pedido_id, cantidad, subtotal FROM detalle_pedido WHERE producto_id = 4 AND eliminado = FALSE ORDER BY pedido_id. ANTES: Seq Scan (descarta 9238 filas, hit=104) + Sort, 0.629 ms. DESPUÉS: Bitmap Heap + Bitmap Index on idx_detalle_producto, 0.208 ms (~3x). Cambio de plan: Sí.

```
CREATE INDEX IF NOT EXISTS idx_detalle_producto
ON detalle_pedido (producto_id)
WHERE eliminado = FALSE;
```

A3 — Q3: productos de categoría 1 ordenados por precio (2003 filas)

Consulta: WHERE categoria_id = 1 AND eliminado = FALSE ORDER BY precio_producto ASC. Índice covering: idx_producto_categoria_precio (categoria_id, precio_producto ASC) INCLUDE (nombre_producto, stock_producto) WHERE eliminado = FALSE. SIN índice: Seq Scan + Sort (quicksort 158 kB), 0.485 ms. CON índice: Index Scan, nodo Sort eliminado, 0.235 ms (~2x). Con catálogo mayor la brecha crece (Sort $O(n \log n)$ vs. orden del índice).

```
CREATE INDEX IF NOT EXISTS idx_producto_categoria_precio
ON producto (categoria_id, precio_producto ASC)
INCLUDE (nombre_producto, stock_producto)
WHERE eliminado = FALSE;
```

A4 — Costo sobre escritura (500 INSERT en detalle_pedido)

Prueba EXPLAIN ANALYZE de 500 INSERT con ROLLBACK posterior. ANTES: 12.275 ms (dominan triggers FK + trg_subtotal + trg_total_ins). DESPUÉS: 13.553 ms (+10.4%). Sobrecarga moderada y aceptable frente a las mejoras de lectura (3x en Q1/Q2b, sin Sort en Q3, 167x en reporte C). Los 3 índices son parciales (solo vigentes), lo que acota el mantenimiento.

Índice descartado por sobreindexación (NO figura en indices.sql)

Se descarta idx_producto_disponible ON producto (disponible): booleana de cardinalidad 2; sin parcial selectivo el optimizador elige Seq Scan igual y solo encarece INSERT/UPDATE. También se descarta idx_pedido_usuario ON pedido (usuario_id) por redundante con idx_pedido_usuario_id de Semana 3. Ver specs/spec_indice_descartado_disponible.md y duia.md fila 3.

3. Parte B — Vistas (views.sql)

B1 — v_reporte_productos_categorias: productos vigentes con su categoría (id, nombre, precio, stock + nombre_categoría; filtra p.eliminado y c.eliminado). B2 — v_seguridad_pedidos_usuarios: pedidos con datos del usuario aplicando seguridad (expone id_pedido, fecha, estado, total, forma_pago + id/nombre/apellido/mail de usuario; OCULTA contraseña_usuario y celular, por lo que se puede otorgar

SELECT sobre la vista sin acceso a la tabla base). B3 — v_reporte_detalle_pedido_producto: detalle con nombre del producto (pedido_id, nombre, cantidad, precio unitario, subtotal).

```
CREATE OR REPLACE VIEW v_seguridad_pedidos_usuarios AS
SELECT ped.id_pedido, ped.fecha_pedido, ped.estado_pedido,
       ped.total_pedido, ped.forma_pago, u.id_usuario,
       u.nombre_usuario, u.apellido_usuario, u.mail_usuario
FROM pedido ped JOIN usuario u ON ped.usuario_id = u.id_usuario
WHERE ped.eliminado = FALSE AND u.eliminado = FALSE;
```

Verificación de equivalencia (EXCEPT bidireccional, mediciones.sql §B)

B1a v EXCEPT manual: 0 filas — B1b manual EXCEPT v: 0 filas. VÁLIDA. B2a: 0 filas — B2b: 0 filas. VÁLIDA. B3a: 0 filas — B3b: 0 filas. VÁLIDA. Seguridad B2: information_schema.columns de la vista NO lista contraseña_usuario (contraseña_expuesta=False).

4. Parte C — Vista materializada

Reporte: facturación por categoría y mes (queries.sql B, 4 JOINS + GROUP BY sobre 10007 filas → 125 grupos). Objeto: mv_facturacion_categoria_mes ... WITH DATA (125 filas) + UNIQUE (id_categoria, mes), que habilita REFRESH CONCURRENTLY.

Consulta original: HashAggregate + 3 Hash Joins (hit=266), 7.027 ms (7.980 / 6.887 / 7.027). SELECT * FROM mv_facturacion_categoria_mes: Seq Scan sobre 125 filas preagregadas (hit=1), 0.042 ms (0.048 / 0.042 / 0.037). Mejora ~167x. REFRESH CONCURRENTLY verificado OK (16.1 ms, 125 filas).

```
CREATE MATERIALIZED VIEW mv_facturacion_categoria_mes AS
SELECT c.id_categoria, c.nombre_categoria,
       date_trunc('month', ped.fecha_pedido) AS mes,
       SUM(dp.subtotal_detallepedido) AS facturacion_total
FROM detalle_pedido dp JOIN pedido ped ON dp.pedido_id = ped.id_pedido
JOIN producto pr ON dp.producto_id = pr.id_producto
JOIN categoria c ON pr.categoria_id = c.id_categoria
WHERE dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY c.id_categoria, c.nombre_categoria, date_trunc('month', ped.fecha_pedido)
WITH DATA;
CREATE UNIQUE INDEX idx_mv_facturacion_unico
ON mv_facturacion_categoria_mes (id_categoria, mes);
```

Frecuencia de refresco

Diaria nocturna tras cierre de caja. El dashboard se lee varias veces al día pero tolera desfase: entre refrescos la MV devuelve datos al cierre de ayer (stale documentado a usuarios); CONCURRENTLY refresca sin bloquear lecturas. Descartado refresco por trigger en cada venta (encarece escritura y anula el beneficio de materializar).

5. DUIA resumida, specs y reproducción

Bitácora completa en duia.md (9 filas: herramienta, prompt/spec, propuesta, decisión). Resumen: filas 1-2 índice Q1 aceptado; fila 3 sobreindexación descartada (decisión humana); filas 4-5 índices Q2/Q3 aceptados; filas 6-7 vistas B1–B3 aceptadas con ajuste menor + EXCEPT 0 filas x6; fila 8 materializada aceptada; fila 9 mediciones reales en PG 16.2 transcriptas. Specs en specs/ (8 archivos, uno por pieza + uno de descarte).

- spec_indice_01_pedido_fecha_estado.md — Q1 (igualdad + rango, criterio: salir de Seq Scan).
- spec_indice_02_detalle_producto.md — Q2 (JOIN + GROUP BY producto_id).
- spec_indice_03_producto_categoria_precio.md — Q3 (filtro + ORDER BY, covering).
- spec_indice_descartado_disponible.md — descarte documentado.

- spec_vista_01/02/03 + spec_vista_materializada_facturacion.md — vistas y MV.

Reproducción (README.md): 1) psql -f schema.sql; 2) objects.sql de Semana 4; 3) data.sql (verificar ~5004 pedidos / ~10007 detalles); 4) mediciones.sql para columna ANTES; 5) indices.sql + views.sql; 6) completar DESPUÉS; 7) REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes. Probar en copia o transacción con ROLLBACK y pg_dump previo.

Regla de la cátedra — Defensa oral

Cada estudiante debe poder explicar sin apoyo de IA cualquier índice o vista: por qué se creó, qué plan cambió, qué costo tiene en escritura y por qué se descartó lo que se descartó. Entregar un script que no se puede explicar equivale a no haber realizado el trabajo.

6. Cierre y estructura entregada

```
Semana5/
|-- schema.sql / data.sql / queries.sql (heredados)
|-- indices.sql (Parte A: 3 índices aceptados)
|-- views.sql (Partes B y C: 3 vistas + 1 materializada + índice único)
|-- specs/ (8 especificaciones Kiro)
|-- mediciones.sql (protocolo reproducible)
|-- informe_mediciones.md (antes/después + escritura + MV)
|-- duia.md (bitácora IA) + README.md (reproducción)
```

Al finalizar, Food Store cuenta con un plan de indexado justificado con datos y el conjunto de vistas que servirá de base para los objetos programables (procedimientos, funciones y disparadores) de la Semana 6. No eliminar ni reescribir estos archivos: se reutilizan en la próxima entrega.